Pierwsze kroki z Burp Suite

Instrukcja jest przeznaczona dla osób, które nie korzystały wcześniej z Burp Suite. Prowadzi od uruchomienia narzędzia do wykrycia ośmiu podatności w przygotowanej aplikacji. Każda z nich powinna zostać opisana w raporcie z testu penetracyjnego.

Po zakończeniu ćwiczenia student potrafi:

- skonfigurować Burp Suite jako proxy dla przeglądarki i odczytać przechwycone żądanie HTTP,
- sporządzić mapę aplikacji na podstawie zakładki *HTTP history*,
- zmodyfikować i ponowić żądanie w *Repeaterze* oraz porównać odpowiedzi,
- rozpoznać błędy uwierzytelniania, kontroli dostępu i logiki biznesowej (rozdziały 8–13),
- opisać znalezisko w formie przyjętej w raporcie z testu penetracyjnego.

Ćwiczenie trwa 90 minut. Oprogramowanie wymienione w rozdziale 0 należy zainstalować przed zajęciami. Orientacyjny podział czasu: rozdziały 0–7 około 30 minut, znaleziska 1–8 około 40 minut, opis znalezisk i pytania kontrolne około 20 minut.

Wynikiem ćwiczenia jest raport zawierający opis ośmiu znalezisk według schematu z rozdziału 14. W trakcie zajęć należy wykonać wszystkie testy, zapisać dowody (żądanie i odpowiedź z *Repeatera*) i opisać w pełni co najmniej dwa znaleziska. Pozostałe opisy należy uzupełnić po zajęciach, w terminie i formie podanych przez prowadzącego.

Test penetracyjny to kontrolowana próba ataku na system, prowadzona w celu wykrycia podatności. Wymaga on pisemnej zgody właściciela systemu, określającej zakres i termin testów. Te same czynności wykonane bez zgody mogą stanowić przestępstwo (art. 267 Kodeksu karnego). Ćwiczenie dotyczy wyłącznie lokalnej aplikacji pod adresem `http://127.0.0.1:8000`, uruchomionej na własnym komputerze. Narzędzia nie należy kierować na inne cele, w szczególności na sieci i systemy AGH.

Spis treści

0. Wymagania 2
1. Czym jest Burp i do czego służy proxy 2
2. Uruchomienie aplikacji 3
3. Pierwsze uruchomienie Burpa 3
4. Konfiguracja przeglądarki 3
5. Przechwytywanie żądania 4
6. Mapa aplikacji: HTTP history 5
7. Repeater 6
8. Znalezisko 1: ciasteczko sesji możliwe do podrobienia 6
9. Znalezisko 2: IDOR (dostęp do cudzych danych) 7
10. Znalezisko 3: eskalacja pionowa (panel administratora) 8
11. Znalezisko 4: mass assignment (zmiana własnej roli) 9
12. Znalezisko 5: cena w adresie URL 10
13. Znaleziska 6–8: logika biznesowa 11
14. Zapis znaleziska w raporcie 12
15. Pytania kontrolne 13

0. Wymagania

Przed zajęciami należy zainstalować następujące oprogramowanie:

- **Python 3.12 lub nowszy**, pobierany ze strony python.org/downloads. W systemie Windows podczas instalacji należy zaznaczyć opcję *Add python.exe to PATH*. Poprawność instalacji sprawdza polecenie `python --version`.
- **Burp Suite Community Edition**, pobierany ze strony portswigger.net/burp/communitydownload. Wersja bezpłatna nie wymaga licencji ani rejestracji.
- **Przeglądarka**: Burp Suite zawiera wbudowaną przeglądarkę Chromium, która wystarcza do ćwiczenia. Jako rozwiązanie zapasowe (rozdział 4) można użyć przeglądarki Firefox, pobieranej ze strony mozilla.org/firefox.
- **Plik aplikacji** `app.py`, udostępniony przez prowadzącego.

Pakiety Pythona potrzebne do uruchomienia aplikacji instaluje się poleceniem opisanym w rozdziale 2. Polecenie to również należy wykonać przed zajęciami.

1. Czym jest Burp i do czego służy proxy

Burp Suite to **proxy HTTP** z zestawem narzędzi do testowania aplikacji webowych. Proxy to pośrednik między przeglądarką a serwerem, przez który przechodzi każde żądanie i każda odpowiedź. Burp zapisuje ten ruch, a po włączeniu przechwytywania pozwala zatrzymać żądanie i zmienić je przed wysłaniem do serwera.

Przeglądarka wysyła żądania na podstawie tego, co udostępnia interfejs aplikacji. Osoba testująca może w Burpie zmienić w żądaniu dowolne dane, także takie, których interfejs nie pozwala edytować. Serwer otrzymuje wyłącznie samo żądanie i nie może ustalić, czy dane pochodzą z interfejsu. Jeśli przyjmuje je bez weryfikacji, w aplikacji występuje podatność.

1.1. Budowa żądania i odpowiedzi HTTP

Żądanie HTTP składa się z wiersza żądania (metoda, ścieżka, wersja protokołu, np. `GET /shop HTTP/1.1`), nagłówków w postaci `Nazwa: wartość`, pustego wiersza i opcjonalnej treści (body). Metoda GET służy do pobierania danych. Formularz HTML wysyłany metodą POST przekazuje dane w treści żądania, domyślnie w postaci `pole1=wartość1&pole2=wartość2`.

Dane kontrolowane przez klienta znajdują się w ścieżce (`/api/orders/1001`), w adresie po znaku `?` (parametry zapytania, np. `?price=1299`), w treści żądania POST oraz w nagłówkach, w tym w nagłówku `Cookie`. Każde z tych miejsc można zmienić w Burpie.

Ciasteczko (cookie) to wartość, którą serwer przekazuje przeglądarce w nagłówku `Set-Cookie`. Przeglądarka dołącza ją do kolejnych żądań w nagłówku `Cookie`. Ciasteczko sesji pozwala serwerowi rozpoznać, który użytkownik wysłał żądanie.

Odpowiedź zaczyna się od wiersza statusu, który zawiera wersję protokołu, kod statusu i jego opis, np. `HTTP/1.1 200 OK`. Kod 200 oznacza powodzenie, 303 przekierowanie na adres podany w nagłówku `Location`, 401 brak uwierzytelnienia (użytkownik nie jest zalogowany), 403 odmowę dostępu zalogowanemu użytkownikowi, a 404 brak zasobu.

Uwierzytelnianie (*authentication*) to ustalenie, kim jest użytkownik, np. na podstawie hasła albo ciasteczka sesji. Kontrola dostępu, nazywana też autoryzacją (*authorization*), to sprawdzenie, czy ustalony użytkownik ma prawo wykonać daną operację na danym obiekcie.

1.2. Narzędzia Burpa używane w ćwiczeniu

- *Proxy* służy do przechwytywania i podglądu ruchu.
- *HTTP history* to chronologiczna lista żądań, które przeszły przez proxy. W ćwiczeniu służy jako uproszczona mapa aplikacji (szerszą mapę, uwzględniającą także odnośniki znalezione w odpowiedziach, buduje podzakładka *Site map* w zakładce *Target*).

- *Repeater* pozwala ponawiać to samo żądanie ze zmianą wybranego parametru. Jest to narzędzie wykorzystywane w ćwiczeniu najczęściej.

Ćwiczenie wykorzystuje Burp Suite Community Edition, czyli wersję bezpłatną. Nie zawiera ona skanera podatności i nie zapisuje projektu na dysk, a *Intruder* (narzędzie do wysyłania zautomatyzowanych serii żądań) działa w niej z ograniczoną szybkością. Żadna z tych funkcji nie jest potrzebna w ćwiczeniu.

2. Uruchomienie aplikacji

W katalogu aplikacji wykonaj polecenia:

```
python -m pip install fastapi uvicorn python-multipart
python -m uvicorn app:app --port 8000
```

Aplikacja nasłuchuje pod adresem `http://127.0.0.1:8000`. Okno terminala z uruchomioną aplikacją należy pozostawić otwarte do końca ćwiczenia. Wszystkie dane są przechowywane w pamięci, dlatego zatrzymanie aplikacji (Ctrl+C) i ponowne uruchomienie przywracają stan początkowy.

Konta testowe:

Login	Hasło	Rola
`alice`	`alice123`	zwykły użytkownik, konto osoby testującej
`bob`	`bob123`	zwykły użytkownik, konto innej osoby
`admin`	`admin123`	administrator

W ćwiczeniu loguje się wyłącznie konto `alice`. Konta `bob` i `admin` są celem testów. Z konta `alice` próbuje się uzyskać ich dane i uprawnienia.

3. Pierwsze uruchomienie Burpa

Uruchom Burp Suite Community Edition. Przy starcie program wyświetla okna wyboru projektu i konfiguracji. W wersji Community wystarczą ustawienia domyślne.

> **KROK**
>
> Na pierwszym ekranie pozostaw *Temporary project in memory* i kliknij *Next*. Na drugim pozostaw *Use Burp defaults* i kliknij *Start Burp*.

Po chwili pojawi się główne okno z zakładkami *Dashboard*, *Target*, *Proxy*, *Intruder*, *Repeater* i kolejnymi. W ćwiczeniu wykorzystywane są zakładki *Proxy* i *Repeater*.

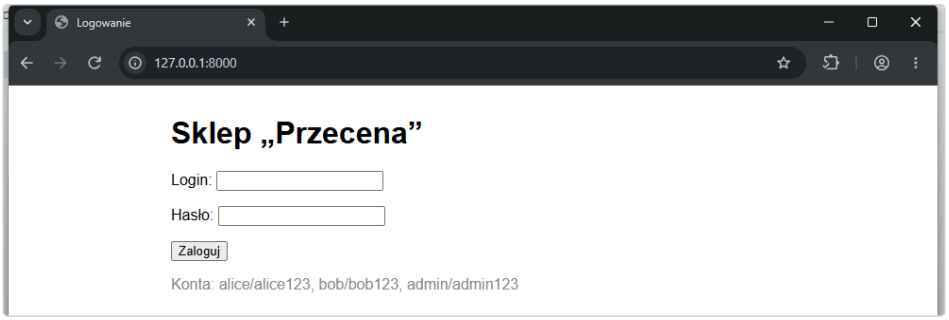
4. Konfiguracja przeglądarki

Ruch przeglądarki musi przechodzić przez Burpa. Najprostszym rozwiązaniem jest wbudowana przeglądarka Chromium, skonfigurowana do korzystania z proxy Burpa. Nie wymaga ona dodatkowych ustawień ani instalacji certyfikatu CA Burpa (certyfikatu potrzebnego do przechwytywania ruchu HTTPS).

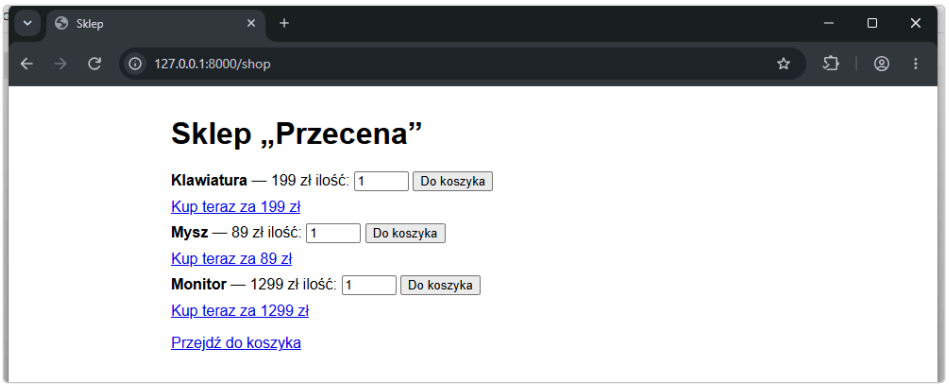
> **KROK**
>
> Otwórz zakładkę *Proxy*, a w niej podzakładkę *Intercept*, i kliknij przycisk *Open browser*. Zostanie otwarte okno przeglądarki. Wprowadź adres `http://127.0.0.1:8000` i zaloguj się jako `alice`.

> **UWAGA**
>
> Jeśli wbudowana przeglądarka się nie uruchomi, otwórz w Burpie okno *Settings*, w sekcji *Tools* wybierz *Burp's browser*, zaznacz opcję *Allow Burp's browser to run without a sandbox* i ponownie kliknij *Open browser*. Jeśli to nie pomoże, użyj Firefoksa. W ustawieniach Firefoksa, na karcie *Ogólne*, w sekcji *Ustawienia sieci* kliknij *Ustawienia…*, wybierz ręczną konfigurację serwerów proxy i wpisz serwer HTTP `127.0.0.1`, port `8080`. Następnie na stronie `about:config` ustaw opcję `network.proxy.allow_hijacking_localhost` na `true`. Bez tej opcji Firefox wysyła żądania do adresu `127.0.0.1` z pominięciem proxy i nie pojawiają się one w Burpie. Aplikacja korzysta z nieszyfrowanego HTTP, więc certyfikat Burpa nie jest potrzebny.

Od tej chwili każde żądanie wysłane przez przeglądarkę przechodzi przez Burpa (zrzuty 1 i 2).



Zrzut 1. Strona logowania aplikacji.

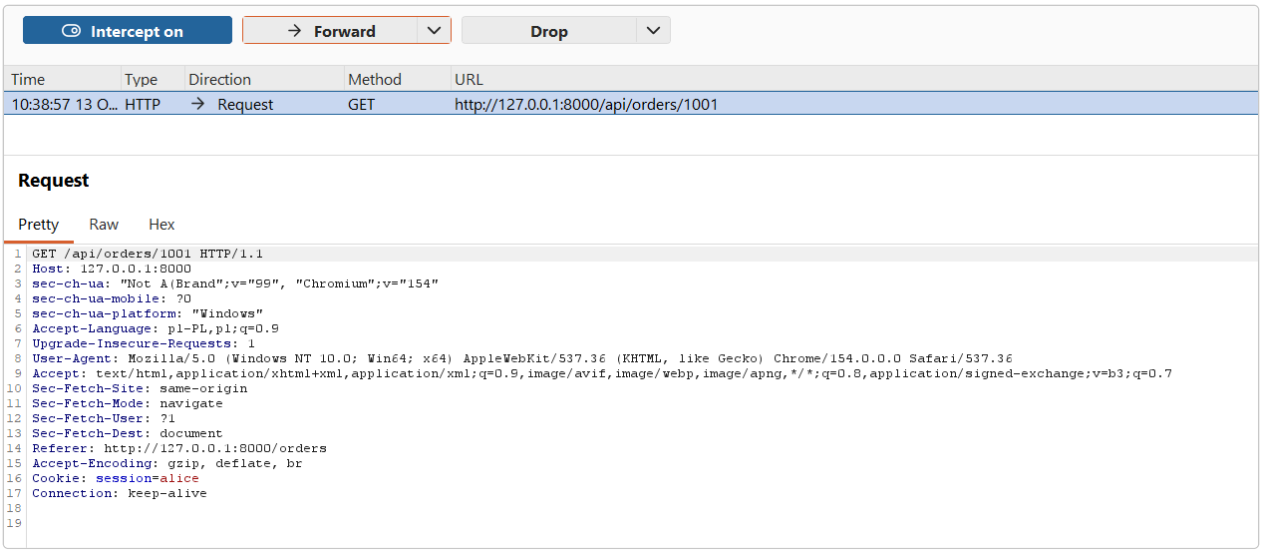


Zrzut 2. Strona sklepu. Przy każdym produkcie znajduje się formularz *Do koszyka* i odnośnik *Kup teraz za X zł*.

5. Przechwytywanie żądania

W zakładce *Proxy*, w podzakładce *Intercept*, znajduje się przełącznik *Intercept off / Intercept on*. Gdy jest włączony, Burp zatrzymuje każde żądanie zgodne z regułami przechwytywania (domyślnie pomijane są m.in. obrazki, pliki CSS i JavaScript) i czeka na decyzję. Przycisk *Forward* przekazuje żądanie do serwera, a *Drop* je odrzuca. Przed podjęciem decyzji żądanie można obejrzeć i zmienić.

> **KROK**
>
> Włącz przechwytywanie (*Intercept on*). W przeglądarce kliknij *Moje zamówienia*. Strona nie zostanie wczytana, ponieważ żądanie zostało zatrzymane w Burpie. Przełącz się na okno Burpa i kliknij *Forward*. Następnie kliknij w przeglądarce zamówienie `#1001`. W zakładce *Intercept* pojawi się żądanie `GET /api/orders/1001`.



Zrzut 3. Przechwycone żądanie `GET /api/orders/1001`. Nagłówek `Cookie: session=alice` pozwala serwerowi rozpoznać użytkownika. Żądanie nie zostało jeszcze wysłane do serwera i oczekuje na decyzję *Forward* albo *Drop*.

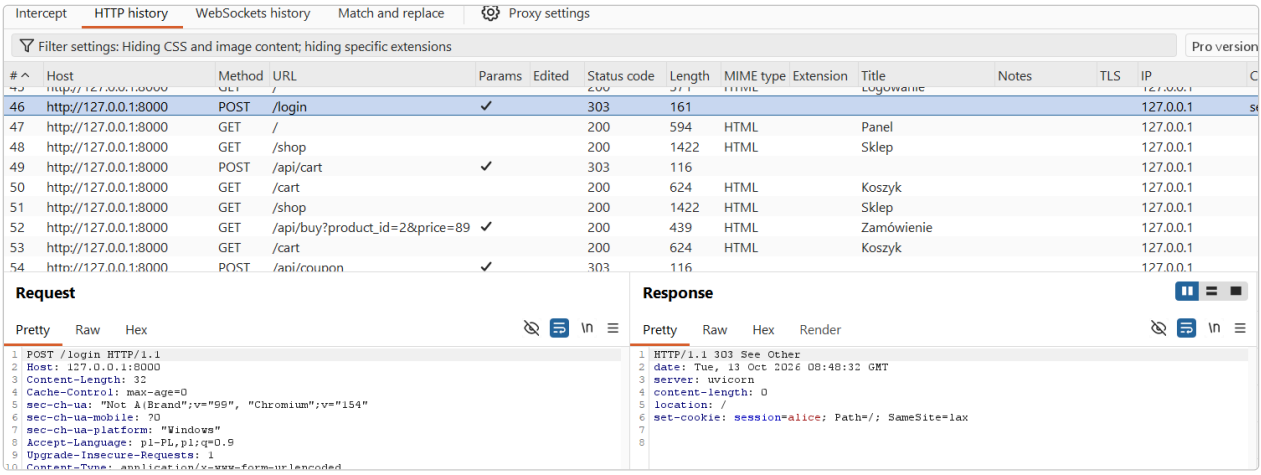
Widoczne są metoda (`GET`), ścieżka (`/api/orders/1001`) i nagłówki, w tym ciasteczko sesji. Kliknij *Forward*, aby przekazać żądanie do serwera.

> **UWAGA**
>
> Każde zatrzymane żądanie wstrzymuje przeglądarkę do czasu decyzji, dlatego *Intercept* zwykle pozostaje wyłączony, a analiza odbywa się w zakładkach *HTTP history* i *Repeater*. *Intercept* włącza się tylko wtedy, gdy trzeba zmienić konkretne żądanie przed wysłaniem.

6. Mapa aplikacji: HTTP history

Wyłącz przechwytywanie (*Intercept off*) i przejdź przez wszystkie podstrony aplikacji, wykonując każdą dostępną akcję z wyjątkiem *Wyloguj*. Dodaj produkt do koszyka (*Do koszyka*), kliknij jeden z odnośników *Kup teraz* przy klawiaturze lub myszy, wpisz w koszyku kod `BSI10` i kliknij *Zastosuj*, zapisz profil (*Zapisz*) i złóż zamówienie (*Zamów*).

Następnie otwórz w zakładce *Proxy* podzakładkę *HTTP history*. Jest to lista wszystkich żądań, które przeszły przez proxy, w kolejności wysłania (najstarsze u góry; kliknięcie nagłówka kolumny `#` odwraca kolejność). Lista pełni rolę mapy aplikacji. Widoczne są na niej endpointy wywołane podczas przeglądania aplikacji (endpoint, czyli punkt końcowy, to metoda i ścieżka obsługiwane przez serwer), kody odpowiedzi i ciasteczka.



Zrzut 4. *HTTP history* po przejściu przez aplikację. Zaznaczone żądanie `POST /login`; w panelu odpowiedzi widoczny nagłówek `Set-Cookie`, który ustawia ciasteczko `session`.

Mapa aplikacji stanowi punkt wyjścia testu. Przejrzyj listę i wypisz endpointy, które przyjmują identyfikator obiektu lub dane z formularza. Przetestuj je w pierwszej kolejności.

Interfejs nie udostępnia wszystkich ścieżek obsługiwanych przez serwer. Ścieżki administracyjne o typowych nazwach (`/admin`, `/administrator`, `/panel`) sprawdza się, wpisując je ręcznie (ang. *forced browsing*).

7. Repeater

Repeater to narzędzie, które pozwala wysłać żądanie, obejrzeć odpowiedź i wysłać to samo żądanie ponownie po dowolnej zmianie. Zwykle zmienia się jeden parametr, wysyła żądanie przyciskiem *Send* i porównuje odpowiedź z poprzednią. W ten sposób wykrywa się większość podatności opisanych w instrukcji.

> **KROK**
>
> W *HTTP history* kliknij żądanie prawym przyciskiem myszy i wybierz *Send to Repeater* (skrót Ctrl+R). Przejdź do zakładki *Repeater*. Po lewej stronie znajduje się żądanie, które można edytować, a po prawej odpowiedź. Przycisk *Send* wysyła żądanie.

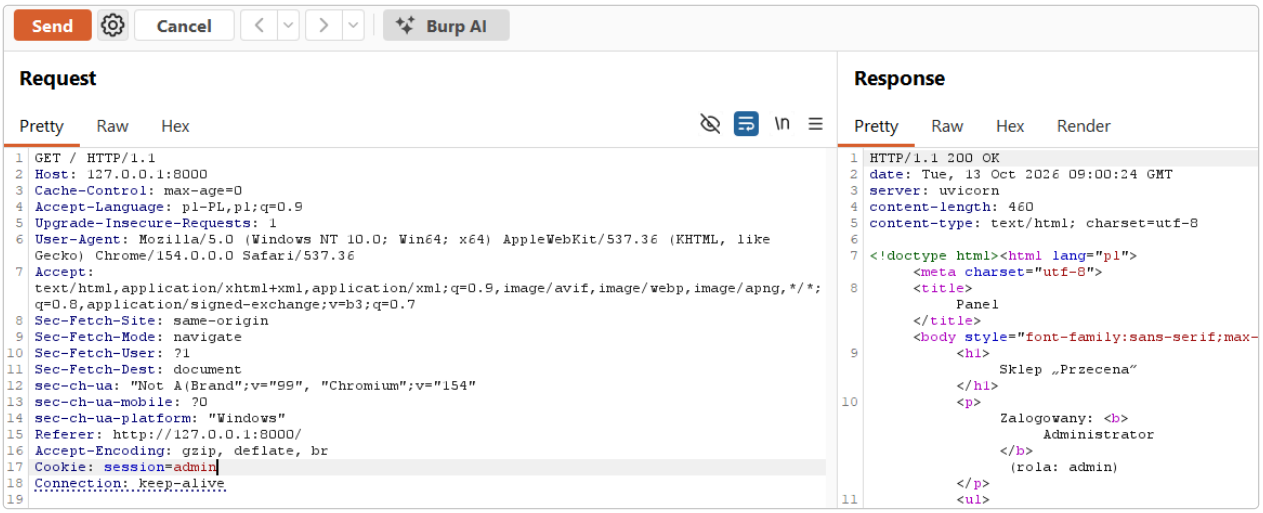
Znalezisko jest potwierdzone, gdy odpowiedź serwera albo stan aplikacji odczytany kolejnym żądaniem potwierdza skutek, np. ujawnienie cudzych danych, wykonanie akcji administratora albo zmianę kwoty zamówienia. Sam kod 200 lub 303 nie jest dowodem.

Zestaw nagłówków na zrzutach może różnić się od widocznego w Burpie. Istotne są ścieżka, parametry i nagłówek `Cookie`. Numery zamówień również mogą być inne niż na zrzutach.

8. Znalezisko 1: ciasteczko sesji możliwe do podrobienia

W *HTTP history* zaznacz żądanie `POST /login` i obejrzyj odpowiedź. Nagłówek ma postać `Set-Cookie: session=alice; Path=/; SameSite=lax`. Wartością ciasteczka sesji jest sam login, zapisany jawnym tekstem i bez podpisu. Serwer nie może więc odróżnić ciasteczka, które sam wydał, od ciasteczka wpisanego ręcznie.

> **KROK**
>
> Wyślij do *Repeatera* żądanie `GET /`. W nagłówku `Cookie` zmień wartość `session=alice` na `session=admin` i kliknij *Send*.



Zrzut 5. Żądanie z nagłówkiem `Cookie: session=admin`. Serwer odpowiada stroną „Zalogowany: Administrator (rola: admin)", choć nie podano hasła administratora.

Zmiana jednej wartości daje dostęp do dowolnego konta bez znajomości hasła. Jest to obejście uwierzytelniania (rozdział 1.1) o wadze krytycznej.

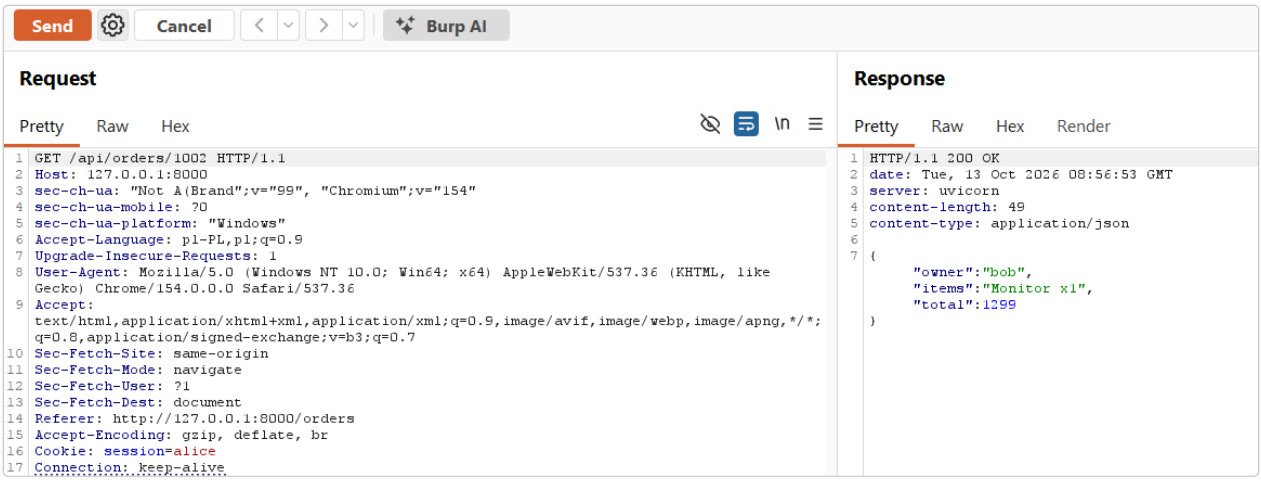
W tym samym nagłówku `Set-Cookie` brakuje dwóch flag. Flaga `HttpOnly` blokuje odczyt ciasteczka przez skrypty JavaScript uruchomione na stronie, co ogranicza skutki ataku XSS (cross-site scripting, czyli wstrzyknięcia skryptu do strony; temat kolejnych wykładów). Flaga `Secure` ogranicza wysyłanie ciasteczka do połączeń HTTPS. Brak tych flag ma niską wagę. Sam w sobie nie umożliwia ataku, ale zwiększa możliwe skutki innych błędów. Atrybut `SameSite=lax` ustawia domyślnie framework aplikacji (Starlette).

> **UWAGA**
>
> W pozostałych testach ciasteczko pozostaje niezmienione (`session=alice`). Test z podmianą ciasteczka dotyczy uwierzytelniania, a pozostałe testy dotyczą kontroli dostępu i logiki aplikacji przy poprawnej sesji.

9. Znalezisko 2: IDOR (dostęp do cudzych danych)

Eskalacja pozioma to dostęp do danych lub funkcji innego użytkownika o tych samych uprawnieniach. Jej najczęstszą przyczyną jest IDOR (*insecure direct object reference*, niebezpieczne bezpośrednie odwołanie do obiektu). Serwer zwraca obiekt wskazany identyfikatorem w żądaniu i nie sprawdza, czy należy on do zalogowanego użytkownika.

> **KROK**
>
> Wyślij do *Repeatera* żądanie `GET /api/orders/1001` (zamówienie konta `alice`). Zmień w ścieżce identyfikator na `1002` i kliknij *Send*.



Zrzut 6. Żądanie zamówienia `1002` wysłane w sesji konta `alice` (`Cookie: session=alice`). Serwer zwraca kod 200 i dane `{"owner":"bob", ...}`. Zamówienie należy do konta `bob`, co potwierdza podatność IDOR.

Jeśli serwer zwróci kod 403 (odmowa dostępu), zasób jest chroniony. Kod 404 może oznaczać brak obiektu albo ochronę, która ukrywa istnienie cudzego obiektu. Aby odróżnić te przypadki, należy użyć identyfikatora obiektu, który na pewno istnieje (zamówienie `1002` należy do konta `bob`). Test należy powtórzyć dla każdego endpointu z mapy aplikacji, który przyjmuje identyfikator obiektu.

> **UWAGA**
>
> Losowy identyfikator, np. UUID w wersji 4 (128-bitowy identyfikator, w którym 122 bity są losowe), utrudnia zgadywanie, ale nie zastępuje autoryzacji. Jeśli taki identyfikator zostanie ujawniony, dostęp do obiektu nadal będzie możliwy.

10. Znalezisko 3: eskalacja pionowa (panel administratora)

Eskalacja pionowa to uzyskanie uprawnień wyższego poziomu, na przykład wywołanie przez zwykłego użytkownika funkcji przeznaczonej dla administratora.

> **KROK**
>
> Jako `alice` otwórz w przeglądarce adres `http://127.0.0.1:8000/admin`.



Zrzut 7. Panel administratora otwarty przez konto `alice` o roli `user`.

Panel zostaje wyświetlony, mimo że konto `alice` nie ma roli administratora. Serwer sprawdza jedynie, czy użytkownik jest zalogowany, i nie weryfikuje jego roli. Ten sam błąd dotyczy endpointu `POST /api/admin/set-role`.

> **KROK**
>
> W panelu przy koncie `bob` wpisz w polu roli `admin` i kliknij *Ustaw rolę*. Lista kont pokaże konto `bob` z rolą `admin`.

Dowodem do raportu są żądania `GET /admin` i `POST /api/admin/set-role` odczytane w *HTTP history*.

> **UWAGA**
>
> Pole roli ma domyślną wartość `user`. Nie klikaj *Ustaw rolę* przy koncie `admin`, ponieważ spowoduje to odebranie temu kontu uprawnień administratora.

11. Znalezisko 4: mass assignment (zmiana własnej roli)

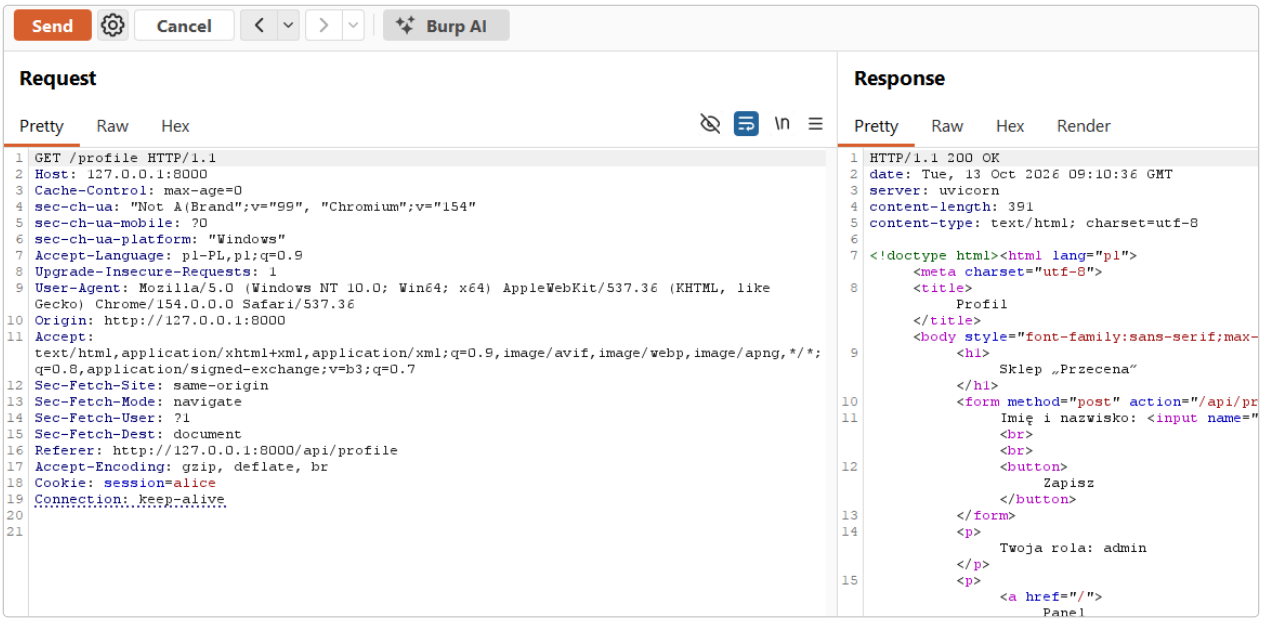
Mass assignment (masowe przypisanie) to błąd, w którym serwer zapisuje do obiektu wszystkie pola przesłane w żądaniu, a nie wyłącznie pola, które użytkownik może zmieniać. Formularz profilu udostępnia tylko pole `name`, a serwer przyjmie także pole `role`.

> **KROK**
>
> Na stronie *Profil* kliknij *Zapisz*. Żądanie `POST /api/profile` pojawi się w *HTTP history*. Wyślij je do *Repeatera*. Treść żądania ma postać `name=Alice+Kowalska` (znak `+` zastępuje spację). Na końcu treści dopisz `&role=admin` i kliknij *Send*. Burp automatycznie aktualizuje nagłówek `Content-Length` (długość treści żądania w bajtach).



Zrzut 8. W treści żądania dopisane pole `role=admin`, którego formularz nie zawiera. Serwer odpowiada kodem 303 i przekierowuje na `/profile`.

Odpowiedzią jest przekierowanie (kod 303), które nie potwierdza skutku zmiany. Kliknij w *Repeaterze* *Follow redirection*, aby zobaczyć stronę profilu po zmianie.



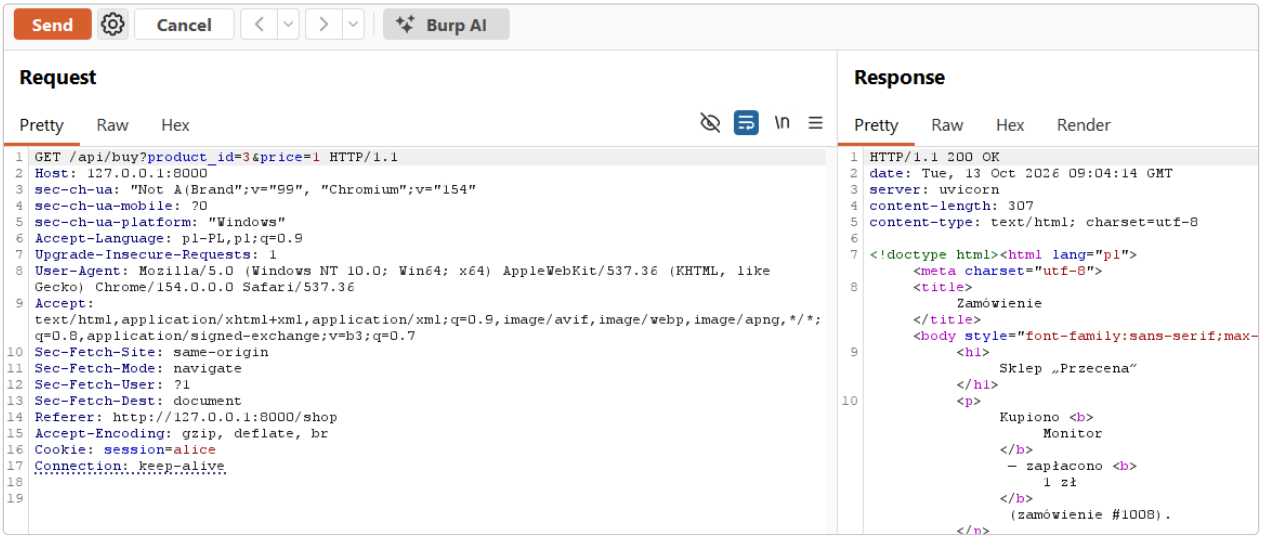
Zrzut 9. Profil po dopisaniu `role=admin`. Strona wyświetla „Twoja rola: admin", czyli zwykły użytkownik nadał sobie uprawnienia administratora.

> **UWAGA**
>
> Po tym teście konto `alice` ma rolę administratora. Przed przejściem dalej przywróć rolę, wysyłając w *Repeaterze* treść `name=Alice+Kowalska&role=user`, albo uruchom aplikację ponownie.

12. Znalezisko 5: cena w adresie URL

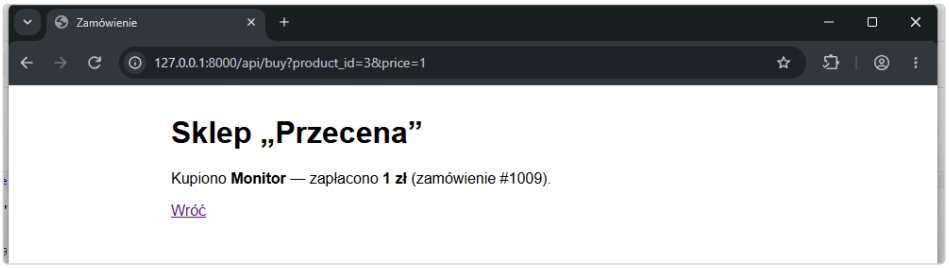
Kliknięcie odnośnika *Kup teraz* wysyła żądanie `GET /api/buy?product_id=3&price=1299`. Cena produktu jest przekazywana w adresie URL, a serwer przyjmuje ją bez porównania z cennikiem.

> **KROK**
>
> W przeglądarce kliknij odnośnik *Kup teraz za 1299 zł* przy monitorze. Żądanie pojawi się w *HTTP history*. Wyślij je do *Repeatera*, zmień `price` na `1` i kliknij *Send*. Każde wysłanie tego żądania tworzy nowe zamówienie.



Zrzut 10. Parametr `price=1` w adresie. Odpowiedź: „Kupiono Monitor — zapłacono 1 zł". Monitor w cenniku kosztuje 1299 zł.

Ten sam efekt można uzyskać bezpośrednio w pasku adresu przeglądarki. Dodatkowo operacja zmieniająca stan aplikacji (zakup) jest wykonywana metodą GET, więc może ją wywołać odnośnik umieszczony na innej stronie.



Zrzut 11. Ten sam atak w pasku adresu przeglądarki. Adres zawiera `price=1`, a aplikacja potwierdza zakup za 1 zł.

13. Znaleziska 6–8: logika biznesowa

Błąd logiki biznesowej to wada reguł działania aplikacji (np. zasad naliczania ceny i rabatu albo kontroli zamawianej ilości), która pozwala użyć poprawnie działających funkcji w sposób niezamierzony przez twórców. Automatyczne skanery zwykle takich błędów nie wykrywają, ponieważ żądania i odpowiedzi są poprawne technicznie. Wykrycie wymaga zrozumienia reguł obowiązujących w aplikacji.

Znaleziska 6 i 7 dotyczą żądania `POST /api/cart`, które przeglądarka wysyła po kliknięciu *Do koszyka*. Już samo kliknięcie dodaje produkt do koszyka, dlatego wartość zmienia się przed wysłaniem żądania, w zakładce *Intercept*. Ponowne wysłanie tego żądania z *Repeatera* dodałoby do koszyka drugą pozycję. Znalezisko 8 polega na powtarzaniu żądania, więc wykonuje się je w *Repeaterze*. Wynik każdego testu widać na stronie *Koszyk* w przeglądarce.

Przed każdym z trzech testów koszyk powinien być pusty. Jeśli pozostały w nim pozycje z wcześniejszego testu, złóż zamówienie przyciskiem *Zamów* (złożenie zamówienia opróżnia koszyk).

13.1. Znalezisko 6: cena z żądania

Formularz *Do koszyka* zawiera ukryte pole `price`, czyli pole formularza niewidoczne na stronie (`<input type="hidden">`), wysyłane razem z pozostałymi polami. Serwer odczytuje cenę z tego pola, a nie z cennika.

> **KROK**
>
> Włącz przechwytywanie (*Intercept on*) i kliknij *Do koszyka* przy monitorze. W zatrzymanym żądaniu `POST /api/cart` zmień w treści wartość `price=1299` na `price=1` i kliknij *Forward*. Wyłącz przechwytywanie (*Intercept off*). Monitor zostanie dodany do koszyka w cenie 1 zł.

Jest to odmiana znaleziska 5, w której cena jest przekazywana w treści żądania POST zamiast w adresie URL.

13.2. Znalezisko 7: ujemna ilość

> **KROK**
>
> W przeglądarce dodaj do koszyka klawiaturę (199 zł). Włącz przechwytywanie (*Intercept on*) i kliknij *Do koszyka* przy monitorze. W zatrzymanym żądaniu `POST /api/cart` zmień `qty=1` na `qty=-1` i kliknij *Forward*. Wyłącz przechwytywanie (*Intercept off*). Suma koszyka wyniesie −1100 zł (199 − 1299). Kliknij *Zamów*, aby złożyć zamówienie na ujemną kwotę.

Serwer nie sprawdza, czy ilość jest dodatnia, i przyjmuje zamówienie o ujemnej wartości (`POST /api/checkout`). Zamówienie jest widoczne na stronie *Moje zamówienia*.

13.3. Znalezisko 8: kupon bez limitu

> **KROK**
>
> Dodaj do koszyka klawiaturę (199 zł). W koszyku wpisz kod `BSI10` (rabat 10%) i kliknij *Zastosuj*. Żądanie `POST /api/coupon` wyślij z *HTTP history* do *Repeatera* i kliknij *Send* jeszcze dwa razy. Po odświeżeniu strony *Koszyk* cena klawiatury wyniesie 144 zł (kolejne naliczenia rabatu dają 179, 161 i 144 zł).



Zrzut 12. Koszyk po trzykrotnym wysłaniu kodu `BSI10` (raz w przeglądarce, dwa razy w *Repeaterze*). Rabat został naliczony przy każdym żądaniu.

Rabat jest naliczany przy każdym żądaniu, ponieważ serwer nie sprawdza, czy kod został już użyty. Rabat obniża cenę wyłącznie produktów, które znajdują się w koszyku w chwili wysłania kodu.

W każdej aplikacji warto sprawdzić wartości, które powinny być dodatnie (ilość, kwota), dane, których klient nie powinien ustalać (cena, rabat, status płatności), oraz operacje, które powinny zostać wykonane jednokrotnie (kupon, przelew).

14. Zapis znaleziska w raporcie

Każde znalezisko opisuje się według jednego schematu:

1. Tytuł i identyfikator, np. „F-02. Dostęp do cudzych zamówień (IDOR)”.
2. Lokalizacja: metoda, endpoint i parametr, np. `GET /api/orders/{id}`, parametr ścieżki `id`.
3. Opis: na czym polega błąd i jaki skutek może wywołać.
4. Kategoria według OWASP Top 10 lub OWASP API Security Top 10 (tabela poniżej). Są to listy najczęstszych kategorii zagrożeń aplikacji webowych i interfejsów API, publikowane przez fundację OWASP. Oznaczenie `A01:2025` to kategoria 1 w wydaniu z 2025 roku.
5. Waga (krytyczna, wysoka, średnia, niska) z jednozdaniowym uzasadnieniem, które opisuje skutek dla właściciela systemu. Zakup dowolnego produktu za 1 zł ma wyższą wagę niż ciasteczko bez flagi `HttpOnly`. Waga krytyczna oznacza przejęcie dowolnego konta lub pełną kontrolę nad danymi bez dodatkowych warunków, wysoka dostęp do cudzych danych lub straty finansowe, średnia skutki ograniczone lub wymagające dodatkowych warunków, a niska osłabienie innych zabezpieczeń bez bezpośredniego skutku.
6. Kroki odtworzenia: numerowana lista czynności, która pozwala innej osobie powtórzyć test.
7. Dowód: żądanie i odpowiedź skopiowane jako tekst z *Repeatera*, skrócone do istotnych wierszy. Zrzut ekranu stanowi uzupełnienie.
8. Rekomendacja: zmiana po stronie serwera, która usuwa przyczynę błędu.

Przykład dowodu dla znaleziska 2:

```
GET /api/orders/1002 HTTP/1.1
Host: 127.0.0.1:8000
Cookie: session=alice

HTTP/1.1 200 OK
```

```
content-type: application/json

{"owner":"bob","items":"Monitor x1","total":1299}
```

Rekomendacja dla tego przykładu: przed zwróceniem zamówienia serwer porównuje pole `owner` z użytkownikiem przypisanym do sesji i w razie niezgodności zwraca kod 404, aby nie ujawniać istnienia obiektu.

Tabela zestawia kategorie i rekomendacje dla wszystkich znalezisk. CWE (Common Weakness Enumeration) to katalog typów słabości oprogramowania; numer CWE wskazuje konkretny typ błędu.

Nr	Kategoria	Rekomendacja
1	A07:2025 Authentication Failures (brak flag: A02:2025 Security Misconfiguration)	losowy identyfikator sesji przechowywany po stronie serwera; flagi `HttpOnly`, `Secure`, `SameSite`
2	A01:2025 Broken Access Control; API1:2023 Broken Object Level Authorization	sprawdzanie właściciela obiektu
3	A01:2025 Broken Access Control; API5:2023 Broken Function Level Authorization	sprawdzanie roli przy każdej akcji administratora
4	A08:2025 Software or Data Integrity Failures; API3:2023 Broken Object Property Level Authorization (CWE-915)	zapisywanie wyłącznie dozwolonych pól
5, 6	A06:2025 Insecure Design	cena odczytywana z cennika po stronie serwera; zakup metodą POST
7	A06:2025 Insecure Design	walidacja ilości (wartość co najmniej 1)
8	A06:2025 Insecure Design (CWE-841)	zapis użycia kodu i sprawdzenie go przed naliczeniem rabatu

Dane innych użytkowników użyte jako dowód należy ograniczyć do minimum, zamaskować w raporcie i usunąć po zakończeniu testu.

15. Pytania kontrolne

1. Dlaczego test IDOR wykonuje się przy niezmienionym ciasteczku sesji?
2. Czym różni się eskalacja pozioma od pionowej? Które znaleziska z ćwiczenia należą do każdej z tych kategorii?
3. Dlaczego ukrycie pola formularza (`type="hidden"`) nie chroni przesyłanej wartości?
4. Które znalezisko ma najwyższą wagę? Uzasadnij odpowiedź, opisując skutek dla właściciela sklepu.
5. Jaka wspólna zasada została naruszona w znaleziskach 5–8? Wskaż dla każdego z nich zmianę po stronie serwera, która usuwa przyczynę błędu.
6. Dlaczego kod odpowiedzi 200 lub 303 nie wystarcza jako dowód znaleziska? Co powinno zostać pokazane zamiast niego?
7. Dlaczego automatyczny skaner podatności nie wykryje kuponu, który można wykorzystać wielokrotnie?

> **DALSZA PRAKTYKA**
>
> Do dalszych ćwiczeń można wykorzystać PortSwigger Web Security Academy (bezpłatne laboratoria przygotowane przez twórców Burpa, z ćwiczeniem i rozwiązaniem dla większości omówionych podatności) oraz aplikacje OWASP Juice Shop i DVWA. Testy należy prowadzić wyłącznie we własnym środowisku lub za pisemną zgodą właściciela systemu.